STL-like Custom Vector Implementation

Theory, Edge Cases & Code Examples

- ✓ What is a Custom Vector?
- ✓ Why Custom Vector Matters in Autonomous Driving
- ✓ Vector vs Array vs List
- ✓ Iterator Invalidation on Reallocation
- ✓ Copy Assignment Exception Safety (Basic Guarantee vs Strong

→ *Swipe to learn* →

STL-like Custom Vector Implementation

Theory

📌 What is a Custom Vector?

A **Custom Vector** is a template class that mimics `std::vector<T>`, providing:
- **Dynamic array behavior**: Automatic growth when capacity is exceeded
- **RAII semantics**: Automatic memory management via constructor/destructor
- **Value semantics**: Proper copy and move constructors/assignments
- **Iterator support**: Range-for loop compatibility and STL algorithm integration
- **Exception safety**: Strong guarantees via copy-and-swap idiom

Key components of a Vector implementation:

```
template <typename T>
class Vector {
    T* data;              // Pointer to heap-allocated array
    std::size_t size;     // Current number of elements
    std::size_t capacity; // Total allocated space
};
```

STL-like Custom Vector Implementation

Theory

📌 Why Custom Vector Matters in Autonomous Driving

In autonomous vehicle software, understanding dynamic containers is crucial for:
1. **LiDAR point cloud buffers**: Storing variable-sized sensor data (100K-2M points per frame)
2. **Object detection results**: Dynamic lists of detected vehicles, pedestrians, traffic signs
3. **Path planning waypoints**: Variable-length trajectory representations
4. **Memory optimization**: Understanding when to use `reserve()` vs `resize()` for real-time performance
5. **Exception safety**: Ensuring sensor data structures remain valid even when memory allocation fails

Performance considerations:
- **Reallocation cost**: Growing from capacity N to 2N requires O(N) copying
- **Amortized O(1) insertion**: Doubling strategy ensures push_back averages constant time
- **Memory overhead**: Typical capacity is 1.5x-2x actual size
- **Cache locality**: Contiguous storage (unlike `std::list`) for better CPU cache performance

STL-like Custom Vector Implementation

Theory

📌 Vector vs Array vs List (1/2)

Feature	Custom Vector	std::array	std::list
Size	Dynamic	Fixed at compile-time	Dynamic
Memory	Contiguous	Contiguous	Non-contiguous (nodes)
Reallocation	Yes (when capacity exceeded)	Never	Never
Random Access	O(1)	O(1)	O(N)
Insertion at end	O(1) amortized	N/A (fixed size)	O(1)
Memory locality	Excellent	Excellent	Poor
Iterator invalidation	On reallocation	Never	Only deleted elements
Use case	Variable-size data	Fixed-size data	Frequent insertion/deletion

Real-world example from autonomous driving:

STL-like Custom Vector Implementation

Theory

📌 Vector vs Array vs List (2/2)

```
// LiDAR point cloud processor
class LidarProcessor {
    Vector<Point3D> point_cloud;  // Dynamic size (varies by frame)

    void processFrame(const LidarData& data) {
        point_cloud.clear();
        point_cloud.reserve(data.expected_points);  // Avoid reallocations

        for (const auto& raw_point : data) {
            Point3D processed = preprocess(raw_point);
            point_cloud.push_back(std::move(processed));  // Move semantics
        }

        // point_cloud now contains 100K-2M points in contiguous memory
        // Excellent cache locality for subsequent algorithms
    }
};
```

STL-like Custom Vector Implementation

Edge Cases

📌 1. Iterator Invalidation on Reallocation (1/2)

When `Vector` reallocates (during `push_back`, `reserve`, `resize`), all existing iterators, pointers, and references to elements become **invalid**. This is a common source of bugs.

```
Vector<int> v = {1, 2, 3};
v.reserve(10);  // capacity = 10, no reallocation yet

int* ptr = &v[1];  // ptr points to element at index 1
auto it = v.begin() + 1;

v.push_back(4);  // No reallocation (capacity sufficient)
std::cout << *ptr;  // ✅ OK: ptr still valid

// Now exceed capacity
for (int i = 5; i < 15; ++i) {
    v.push_back(i);  // ❌ Reallocation happens at i=11
}

// After reallocation:
std::cout << *ptr;  // ❌ UNDEFINED BEHAVIOR: ptr points to freed memory
std::cout << *it;   // ❌ UNDEFINED BEHAVIOR: iterator invalidated
```

Why it happens: Reallocation involves:

1. Allocate new larger buffer (`new T[new_capacity]`)
2. Copy/move elements from old buffer to new buffer
3. Delete old buffer (`delete[] old_data`)
4. All pointers to old buffer now point to freed memory

Real-world autonomous vehicle impact:

```
// ❌ BUG: Processing point cloud with invalidated iterators
void processLidar(Vector<Point3D>& cloud) {
    auto it = cloud.begin();

    for (size_t i = 0; i < cloud.size(); ++i) {
        if (needsFiltering(cloud[i])) {
            cloud.push_back(interpolate(cloud[i]));  // May reallocate!
            // 'it' is now invalid if reallocation occurred
        }
// ...continues on next slide
```

STL-like Custom Vector Implementation

Edge Cases

📌 1. Iterator Invalidation on Reallocation (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        ++it;  // ❌ CRASH: dereferencing invalidated iterator
    }
}

// ✅ FIX: Reserve capacity or use index-based access
void processLidarSafe(Vector<Point3D>& cloud) {
    size_t original_size = cloud.size();
    cloud.reserve(original_size * 2);  // Prevent reallocation

    for (size_t i = 0; i < original_size; ++i) {
        if (needsFiltering(cloud[i])) {
            cloud.push_back(interpolate(cloud[i]));  // No reallocation
        }
    }
}
```

Key takeaway: Always call `reserve()` before a loop that may insert elements, or use index-based access instead of iterators.

STL-like Custom Vector Implementation

Edge Cases

📌 2. Copy Assignment Exception Safety (Basic Guarantee vs Strong Guarantee) (1/2)

A naive copy assignment operator can leave the object in a **broken state** if an exception is thrown during copying:

```
// ❌ UNSAFE: Basic exception guarantee (broken state possible)
Vector<T>& operator=(const Vector<T>& other) {
    if (this != &other) {
        delete[] data;  // Delete old data first
        size = other.size;
        capacity = other.capacity;
        data = new T[capacity];  // ✅ May throw (allocation failure)

        for (size_t i = 0; i < size; ++i) {
            data[i] = other.data[i];  // ❌ May throw (T's copy throws)
        }
    }
    return *this;
}

// Problem scenario:
struct ExpensiveObject {
    ExpensiveObject(const ExpensiveObject& other) {
        if (rand() % 10 == 0) throw std::runtime_error("copy failed");
        // ... copying logic ...
    }
};

Vector<ExpensiveObject> v1 = {obj1, obj2, obj3};
Vector<ExpensiveObject> v2 = {obj4, obj5};

v1 = v2;  // ❌ If copy throws at i=2:
          //    - v1.data is deleted (old data gone)
          //    - new allocation succeeded
          //    - first 2 elements copied
          //    - exception thrown at element 3
          //    - v1 now has partially-copied data (BROKEN STATE!)
          //    - v1.size=3 but only 2 elements valid
```

Strong exception guarantee solution (copy-and-swap idiom):

STL-like Custom Vector Implementation

Edge Cases

📌 2. Copy Assignment Exception Safety (Basic Guarantee vs Strong Guarantee) (2/2)

```
// ✅ SAFE: Strong exception guarantee (all-or-nothing)
Vector<T>& operator=(const Vector<T>& other) {
    if (this != &other) {
        Vector<T> temp(other);  // ✅ May throw, but *this unchanged
        swap(temp);             // ❌ Won't throw (just pointer swaps)
    }
    return *this;
}
// When temp goes out of scope, destructor cleans up old data

void swap(Vector<T>& other) noexcept {
    std::swap(data, other.data);
    std::swap(size, other.size);
    std::swap(capacity, other.capacity);
}
```

Why this is better:

1. `Vector<T> temp(other)` does all risky operations (allocation, copying)
2. If temp construction throws → `*this` remains **unchanged** (strong guarantee)
3. If temp construction succeeds → `swap()` just exchanges pointers (noexcept)
4. temp's destructor cleans up old data

Key takeaway: Use copy-and-swap idiom for assignment operators to achieve strong exception safety.

STL-like Custom Vector Implementation

Edge Cases

📌 3. Return-by-Reference vs Return-by-Value for Iterators (1/2)

A common mistake when implementing `begin()` and `end()` is returning references to **local variables**:

```
// ❌ BUG: Returning reference to local variable
Iterator begin() {
    Iterator it(data);  // 'it' is a local variable on the stack
    return it;          // ❌ Returns by value (correct!)
}

// ❌ CRITICAL BUG: Returning reference to local
Iterator& begin() {
    Iterator it(data);  // 'it' is destroyed when function returns
    return it;          // ❌ UNDEFINED BEHAVIOR: dangling reference!
}

// Usage:
Vector<int> v = {1, 2, 3};
auto it = v.begin();  // ❌ 'it' refers to destroyed local variable
std::cout << *it;     // ❌ CRASH or garbage value
```

Correct implementation (return by value):

```
// ✅ Correct: Return by value
Iterator begin() { return Iterator(data); }
Iterator end()   { return Iterator(data + size); }

Const_Iterator cbegin() const { return Const_Iterator(data); }
Const_Iterator cend()   const { return Const_Iterator(data + size); }

// Iterator is lightweight (just a pointer wrapper), so returning by value is fine
class Iterator {
    T* ptr;  // 8 bytes on 64-bit systems
public:
    Iterator(T* p) : ptr(p) {}
    // Cheap to copy
};
```

STL-like Custom Vector Implementation

Edge Cases

📌 3. Return-by-Reference vs Return-by-Value for Iterators (2/2)

Why return by value is correct:
- Iterators are lightweight (typically just a pointer)
- Returning by value creates a copy, avoiding dangling references
- RVO (Return Value Optimization) often eliminates the copy anyway
- Local variable `it` is destroyed, but the **returned copy** is valid

Key takeaway: Always return iterators by value, never by reference to locals.

STL-like Custom Vector Implementation

Edge Cases

📌 4. Move Constructor and Moved-From State (1/2)

After move construction/assignment, the **moved-from object** must remain in a **valid but unspecified state**:

```
// ✅ Correct move constructor
Vector(Vector<T>&& other) noexcept
    : data(nullptr), size(0), capacity(0)
{
    swap(other);  // Exchange internals
}

// After move:
Vector<int> v1 = {1, 2, 3};
Vector<int> v2 = std::move(v1);

// v1 is now in "moved-from" state:
// - v1.data = nullptr
// - v1.size = 0
// - v1.capacity = 0
// ✅ v1 can still be destroyed safely (destructor handles nullptr)
// ✅ v1 can be reassigned: v1 = {4, 5, 6};
// ❌ v1[0] is undefined behavior (size is 0)
```

Common bug: Not resetting moved-from object:

```
// ❌ BUG: Moved-from object not in valid state
Vector(Vector<T>&& other) noexcept {
    data = other.data;
    size = other.size;
    capacity = other.capacity;
    // ❌ Forgot to reset other.data to nullptr!
}

// Destructor of moved-from object:
~Vector() {
    delete[] data;  // ❌ DOUBLE DELETE! Both objects delete same memory
}
```

Correct implementation:

STL-like Custom Vector Implementation

Edge Cases

📌 4. Move Constructor and Moved-From State (2/2)

```
// ✅ Using swap (automatic reset)
Vector(Vector<T>&& other) noexcept
    : data(nullptr), size(0), capacity(0)
{
    swap(other);  // other gets nullptr, 0, 0
}

// ✅ Manual reset
Vector(Vector<T>&& other) noexcept
    : data(other.data), size(other.size), capacity(other.capacity)
{
    other.data = nullptr;
    other.size = 0;
    other.capacity = 0;
}
```

Real-world implication (autonomous driving):

```
// Sensor data processing with move semantics
Vector<Point3D> processLidar(Vector<Point3D> raw_cloud) {
    Vector<Point3D> filtered;
    filtered.reserve(raw_cloud.size());

    for (const auto& pt : raw_cloud) {
        if (isValid(pt)) filtered.push_back(pt);
    }

    return filtered;  // Move semantics (RVO or move constructor)
}

// Usage:
Vector<Point3D> cloud = getLidarData();
Vector<Point3D> processed = processLidar(std::move(cloud));
// 'cloud' is moved-from but still valid (can be reassigned or destroyed)
```

Key takeaway: Always reset moved-from objects to a valid state (nullptr for pointers, 0 for sizes).

STL-like Custom Vector Implementation

Edge Cases

📌 5. Reserve vs Resize Semantics (1/2)

Confusing `reserve()` and `resize()` leads to bugs because they have **different effects** on size:

```
Vector<int> v;

// reserve(): Changes capacity, NOT size
v.reserve(100);
std::cout << v.size();      // 0 (no elements constructed)
std::cout << v.capacity();  // 100 (space allocated)
v[50] = 42;  // ❌ UNDEFINED BEHAVIOR: index out of bounds (size is 0!)

// resize(): Changes size (constructs/destructs elements)
v.resize(100);
std::cout << v.size();      // 100 (elements default-constructed)
std::cout << v.capacity();  // ≥100 (at least 100)
v[50] = 42;  // ✅ OK: element exists
```

reserve() behavior:

```
void reserve(size_t new_cap) {
    if (new_cap <= capacity) return;  // No-op if sufficient

    T* new_data = new T[new_cap];  // Allocate larger buffer
    for (size_t i = 0; i < size; ++i) {
        new_data[i] = std::move(data[i]);  // Move existing elements
    }
    delete[] data;
    data = new_data;
    capacity = new_cap;
    // size UNCHANGED
}
```

resize() behavior:

```
void resize(size_t new_size, const T& val = T{}) {
    if (new_size < size) {
        size = new_size;  // Shrink (no reallocation)
    } else if (new_size > size) {
        if (new_size > capacity) reserve(new_size);  // Grow capacity if needed
        for (size_t i = size; i < new_size; ++i) {
// ...continues on next slide
```

STL-like Custom Vector Implementation

Edge Cases

📌 5. Reserve vs Resize Semantics (2/2)

[Code continues across slides]

```
// ...continued from previous slide
            data[i] = val;  // Default-construct new elements
        }
        size = new_size;
    }
}
```

Real-world scenario (autonomous driving):

```
// Preallocating LiDAR point buffer
class LidarBuffer {
    Vector<Point3D> points;

    void prepareForFrame(size_t expected_point_count) {
        // ✅ Use reserve() to avoid reallocations during insertion
        points.clear();
        points.reserve(expected_point_count);
        // size = 0, capacity = expected_point_count
        // Ready for push_back() without reallocation
    }

    void initializeWithDefaults(size_t point_count) {
        // ✅ Use resize() to create default-constructed points
        points.resize(point_count);  // Creates point_count elements
        // size = point_count, all elements initialized to Point3D{}

        // Now can assign directly:
        for (size_t i = 0; i < point_count; ++i) {
            points[i] = readSensorData(i);
        }
    }
};
```

Key takeaway: Use `reserve()` to optimize insertion performance (avoid reallocations), use `resize()` to actually create elements.

STL-like Custom Vector Implementation

Code Examples

📌 1. Basic Vector Implementation (Easy Level) (1/3)

This basic implementation demonstrates:
- RAII: Memory automatically freed in destructor
- Dynamic growth: Capacity doubles when full (amortized O(1) insertion)
- Deep copy: Copy constructor copies all elements
- No move semantics yet (covered in mid-level)

Real-world usage: Storing variable-sized sensor data frames where size is unknown at compile time.

```
#include <iostream>
#include <cstddef>

// Minimal Vector implementation with dynamic growth
template <typename T>
class Vector {
private:
    T* data;
    std::size_t size;
    std::size_t capacity;

public:
    // Constructor
    Vector() : data(nullptr), size(0), capacity(0) {}

    // Destructor (RAII cleanup)
    ~Vector() {
        delete[] data;
    }

    // Copy constructor
    Vector(const Vector<T>& other)
        : data(new T[other.capacity]),
          size(other.size),
          capacity(other.capacity)
    {
        for (std::size_t i = 0; i < size; ++i) {
            data[i] = other.data[i];  // Deep copy
        }
    }
// ...continues on next slide
```

STL-like Custom Vector Implementation

Code Examples

📌 1. Basic Vector Implementation (Easy Level) (2/3)

```
// ...continued from previous slide

    // Copy assignment
    Vector<T>& operator=(const Vector<T>& other) {
        if (this != &other) {
            delete[] data;
            size = other.size;
            capacity = other.capacity;
            data = new T[capacity];
            for (std::size_t i = 0; i < size; ++i) {
                data[i] = other.data[i];
            }
        }
        return *this;
    }

    // Push back with automatic growth
    void push_back(const T& value) {
        if (size == capacity) {
            // Double capacity (or 1 if empty)
            std::size_t new_cap = (capacity == 0) ? 1 : capacity * 2;
            T* new_data = new T[new_cap];

            // Copy existing elements
            for (std::size_t i = 0; i < size; ++i) {
                new_data[i] = data[i];
            }

            delete[] data;
            data = new_data;
            capacity = new_cap;
        }
        data[size++] = value;
    }

    // Element access
    T& operator[](std::size_t index) { return data[index]; }
    const T& operator[](std::size_t index) const { return data[index]; }

    std::size_t get_size() const { return size; }
    std::size_t get_capacity() const { return capacity; }
// ...continues on next slide
```

STL-like Custom Vector Implementation

Code Examples

📌 1. Basic Vector Implementation (Easy Level) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    bool empty() const { return size == 0; }
};

// Usage example: Simple particle system
struct Particle {
    float x, y, vx, vy;
};

int main() {
    Vector<Particle> particles;

    // Add particles
    for (int i = 0; i < 5; ++i) {
        particles.push_back({float(i), float(i*2), 1.0f, -1.0f});
    }

    std::cout << "Particle count: " << particles.get_size() << "\n";
    std::cout << "Capacity: " << particles.get_capacity() << "\n";
    // Output: Particle count: 5, Capacity: 8 (doubled from 4)

    return 0;
}
```

STL-like Custom Vector Implementation

Code Examples

📌 2. Vector with Reserve and Move Semantics (Mid Level) (1/4)

This mid-level implementation adds:
- **Move semantics**: Efficient transfer of resources (no deep copy)
- **reserve()**: Preallocate capacity to avoid reallocations
- **resize()**: Change size, construct/destruct elements
- **Copy-and-swap idiom**: Exception-safe assignment
- **Move overload for push_back**: Optimize temporary insertion

Performance benefit: Moving a `CameraFrame` (3 pointer swaps) vs copying (copying 6MB pixel buffer).

```
#include <iostream>
#include <utility>  // std::move, std::swap

template <typename T>
class Vector {
private:
    T* data;
    std::size_t size;
    std::size_t capacity;

public:
    Vector() : data(nullptr), size(0), capacity(0) {}

    ~Vector() { delete[] data; }

    // Copy constructor
    Vector(const Vector<T>& other)
        : data(nullptr), size(0), capacity(0)
    {
        reserve(other.size);
        for (std::size_t i = 0; i < other.size; ++i) {
            data[i] = other.data[i];
        }
        size = other.size;
    }

    // Move constructor (steal resources)
    Vector(Vector<T>&& other) noexcept
        : data(nullptr), size(0), capacity(0)
// ...continues on next slide
```

STL-like Custom Vector Implementation

Code Examples

📌 2. Vector with Reserve and Move Semantics (Mid Level) (2/4)

```
// ...continued from previous slide
    {
        std::swap(data, other.data);
        std::swap(size, other.size);
        std::swap(capacity, other.capacity);
        // 'other' now has nullptr, 0, 0
    }

    // Copy assignment (copy-and-swap idiom)
    Vector<T>& operator=(const Vector<T>& other) {
        if (this != &other) {
            Vector<T> temp(other);  // May throw
            std::swap(data, temp.data);
            std::swap(size, temp.size);
            std::swap(capacity, temp.capacity);
            // temp destructor cleans up old data
        }
        return *this;
    }

    // Move assignment
    Vector<T>& operator=(Vector<T>&& other) noexcept {
        if (this != &other) {
            std::swap(data, other.data);
            std::swap(size, other.size);
            std::swap(capacity, other.capacity);
        }
        return *this;
    }

    // Reserve capacity (no size change)
    void reserve(std::size_t new_cap) {
        if (new_cap <= capacity) return;

        T* new_data = new T[new_cap];
        for (std::size_t i = 0; i < size; ++i) {
            new_data[i] = std::move(data[i]);  // Move semantics
        }
        delete[] data;
        data = new_data;
        capacity = new_cap;
// ...continues on next slide
```

STL-like Custom Vector Implementation

Code Examples

📌 2. Vector with Reserve and Move Semantics (Mid Level) (3/4)

```
// ...continued from previous slide
    }

    // Resize (change size, construct/destruct elements)
    void resize(std::size_t new_size, const T& val = T{}) {
        if (new_size > capacity) reserve(new_size);

        if (new_size > size) {
            // Construct new elements
            for (std::size_t i = size; i < new_size; ++i) {
                data[i] = val;
            }
        }
        size = new_size;
    }

    // Push back with move overload
    void push_back(const T& value) {
        if (size == capacity) reserve(capacity == 0 ? 1 : capacity * 2);
        data[size++] = value;
    }

    void push_back(T&& value) {
        if (size == capacity) reserve(capacity == 0 ? 1 : capacity * 2);
        data[size++] = std::move(value);  // Move instead of copy
    }

    T& operator[](std::size_t index) { return data[index]; }
    const T& operator[](std::size_t index) const { return data[index]; }

    std::size_t get_size() const { return size; }
    std::size_t get_capacity() const { return capacity; }
    bool empty() const { return size == 0; }

    T& back() { return data[size - 1]; }
    void pop_back() { --size; }
};

// Autonomous driving: Camera frame buffer
struct CameraFrame {
    uint8_t* pixels;  // Large image data
// ...continues on next slide
```

STL-like Custom Vector Implementation

Code Examples

📌 2. Vector with Reserve and Move Semantics (Mid Level) (4/4)

[Code continues across slides]

```
// ...continued from previous slide
    size_t width, height;

    CameraFrame() : pixels(nullptr), width(0), height(0) {}

    // Move constructor (avoid copying large pixel buffer)
    CameraFrame(CameraFrame&& other) noexcept
        : pixels(other.pixels), width(other.width), height(other.height)
    {
        other.pixels = nullptr;
    }

    ~CameraFrame() { delete[] pixels; }
};

int main() {
    Vector<CameraFrame> frame_buffer;

    // Reserve space to avoid reallocations
    frame_buffer.reserve(60);  // 60 fps buffer

    for (int i = 0; i < 60; ++i) {
        CameraFrame frame;
        frame.width = 1920;
        frame.height = 1080;
        frame.pixels = new uint8_t[1920 * 1080 * 3];  // RGB

        // Move frame into buffer (no copy of large pixel data)
        frame_buffer.push_back(std::move(frame));
    }

    std::cout << "Frames buffered: " << frame_buffer.get_size() << "\n";
    // Output: Frames buffered: 60 (no reallocations due to reserve)

    return 0;
}
```

STL-like Custom Vector Implementation

Code Examples

📌 3. Vector with Iterators (Advanced Level) (1/5)

This advanced implementation adds:
- **Random access iterators**: Full STL iterator traits
- **Range-for support**: `begin()` and `end()` enable `for (auto x : vec)`
- **STL algorithm compatibility**: Works with `std::sort`, `std::find`, `std::copy`, etc.
- **Const iterators**: For read-only traversal
- **Iterator arithmetic**: `it + 5`, `it - 3`, `it2 - it1`

Real-world benefit: Seamless integration with STL algorithms for object detection sorting, filtering, and processing.

```
#include <iostream>
#include <iterator>  // std::iterator_traits
#include <algorithm> // std::sort

template <typename T>
class Vector {
private:
    T* data;
    std::size_t size;
    std::size_t capacity;

public:
    // ... constructors, destructor, etc. (same as before) ...

    Vector() : data(nullptr), size(0), capacity(0) {}
    ~Vector() { delete[] data; }

    // Iterator implementation
    class Iterator {
    private:
        T* ptr;
    public:
        // Iterator traits (required for STL compatibility)
        using difference_type = std::ptrdiff_t;
        using value_type = T;
        using pointer = T*;
        using reference = T&;
        using iterator_category = std::random_access_iterator_tag;

// ...continues on next slide
```

STL-like Custom Vector Implementation

Code Examples

📌 3. Vector with Iterators (Advanced Level) (2/5)

```
// ...continued from previous slide
        explicit Iterator(T* p = nullptr) : ptr(p) {}

        // Dereference operators
        T& operator*() const { return *ptr; }
        T* operator->() const { return ptr; }

        // Increment/decrement
        Iterator& operator++() { ++ptr; return *this; }
        Iterator operator++(int) { Iterator tmp = *this; ++ptr; return tmp; }
        Iterator& operator--() { --ptr; return *this; }
        Iterator operator--(int) { Iterator tmp = *this; --ptr; return tmp; }

        // Arithmetic operators (random access)
        Iterator operator+(difference_type n) const { return Iterator(ptr + n); }
        Iterator operator-(difference_type n) const { return Iterator(ptr - n); }
        difference_type operator-(const Iterator& other) const { return ptr - other.ptr; }

        // Comparison operators
        bool operator==(const Iterator& other) const { return ptr == other.ptr; }
        bool operator!=(const Iterator& other) const { return ptr != other.ptr; }
        bool operator<(const Iterator& other) const { return ptr < other.ptr; }
        bool operator>(const Iterator& other) const { return ptr > other.ptr; }

        // Subscript operator
        T& operator[](difference_type n) const { return ptr[n]; }
    };

    // Const iterator
    class ConstIterator {
    private:
        const T* ptr;
    public:
        using difference_type = std::ptrdiff_t;
        using value_type = T;
        using pointer = const T*;
        using reference = const T&;
        using iterator_category = std::random_access_iterator_tag;

        explicit ConstIterator(const T* p = nullptr) : ptr(p) {}

// ...continues on next slide
```

STL-like Custom Vector Implementation

Code Examples

📌 3. Vector with Iterators (Advanced Level) (3/5)

```
// ...continued from previous slide
        const T& operator*() const { return *ptr; }
        const T* operator->() const { return ptr; }

        ConstIterator& operator++() { ++ptr; return *this; }
        ConstIterator operator++(int) { ConstIterator tmp = *this; ++ptr; return tmp; }

        bool operator==(const ConstIterator& other) const { return ptr == other.ptr; }
        bool operator!=(const ConstIterator& other) const { return ptr != other.ptr; }
    };

    // Iterator access methods
    Iterator begin() { return Iterator(data); }
    Iterator end() { return Iterator(data + size); }

    ConstIterator begin() const { return ConstIterator(data); }
    ConstIterator end() const { return ConstIterator(data + size); }

    ConstIterator cbegin() const { return ConstIterator(data); }
    ConstIterator cend() const { return ConstIterator(data + size); }

    // ... other methods ...
    void push_back(const T& value) {
        if (size == capacity) {
            std::size_t new_cap = capacity == 0 ? 1 : capacity * 2;
            T* new_data = new T[new_cap];
            for (std::size_t i = 0; i < size; ++i) {
                new_data[i] = std::move(data[i]);
            }
            delete[] data;
            data = new_data;
            capacity = new_cap;
        }
        data[size++] = value;
    }

    T& operator[](std::size_t idx) { return data[idx]; }
    std::size_t get_size() const { return size; }
};

// Autonomous driving: Object detection with STL algorithms
// ...continues on next slide
```

STL-like Custom Vector Implementation

Code Examples

📌 3. Vector with Iterators (Advanced Level) (4/5)

```
// ...continued from previous slide
struct DetectedObject {
    float distance;
    float confidence;
    std::string type;  // "car", "pedestrian", "bike"
};

int main() {
    Vector<DetectedObject> objects;

    objects.push_back({15.2f, 0.95f, "car"});
    objects.push_back({8.5f, 0.89f, "pedestrian"});
    objects.push_back({22.1f, 0.78f, "bike"});
    objects.push_back({5.3f, 0.98f, "car"});

    // Range-for loop (enabled by begin/end)
    std::cout << "All detected objects:\n";
    for (const auto& obj : objects) {
        std::cout << obj.type << " at " << obj.distance << "m\n";
    }

    // STL algorithm usage (enabled by random access iterators)
    std::sort(objects.begin(), objects.end(),
        [](const DetectedObject& a, const DetectedObject& b) {
            return a.distance < b.distance;  // Sort by distance
        });

    std::cout << "\nSorted by distance:\n";
    for (const auto& obj : objects) {
        std::cout << obj.type << " at " << obj.distance << "m\n";
    }

    // Output:
    // All detected objects:
    // car at 15.2m
    // pedestrian at 8.5m
    // bike at 22.1m
    // car at 5.3m
    //
    // Sorted by distance:
    // car at 5.3m
// ...continues on next slide
```

STL-like Custom Vector Implementation

Code Examples

📌 3. Vector with Iterators (Advanced Level) (5/5)

[Code continues across slides]

```
// ...continued from previous slide
    // pedestrian at 8.5m
    // car at 15.2m
    // bike at 22.1m

    return 0;
}
```

STL-like Custom Vector Implementation

Code Examples

📌 4. Exception-Safe Vector with Copy-and-Swap (Advanced) (1/4)

This demonstrates:
- **Strong exception guarantee**: Copy assignment uses copy-and-swap idiom
- **Exception handling in copy constructor**: Try-catch with cleanup
- **Moved-from state validity**: Objects can still be destroyed after move
- **Testing exception safety**: Simulating copy failures

Key insight: Copy-and-swap ensures that if copying fails, the original object (`*this`) remains **unchanged**. Without this idiom, `*this` could be left in a **broken state** (partially copied, invalid pointers).

```
#include <iostream>
#include <stdexcept>
#include <utility>

// Type that throws on copy (for testing exception safety)
struct ThrowOnCopy {
    int value;
    static int copy_count;

    ThrowOnCopy(int v = 0) : value(v) {}

    ThrowOnCopy(const ThrowOnCopy& other) : value(other.value) {
        if (++copy_count == 3) {
            throw std::runtime_error("Simulated copy failure");
        }
    }

    ThrowOnCopy& operator=(const ThrowOnCopy& other) {
        if (++copy_count == 3) {
            throw std::runtime_error("Simulated assignment failure");
        }
        value = other.value;
        return *this;
    }
};
int ThrowOnCopy::copy_count = 0;

template <typename T>
class Vector {
// ...continues on next slide
```

STL-like Custom Vector Implementation

Code Examples

📌 4. Exception-Safe Vector with Copy-and-Swap (Advanced) (2/4)

```
// ...continued from previous slide
private:
    T* data;
    std::size_t size;
    std::size_t capacity;

    void swap(Vector<T>& other) noexcept {
        std::swap(data, other.data);
        std::swap(size, other.size);
        std::swap(capacity, other.capacity);
    }

public:
    Vector() : data(nullptr), size(0), capacity(0) {}

    ~Vector() { delete[] data; }

    // Copy constructor with exception safety
    Vector(const Vector<T>& other) : data(nullptr), size(0), capacity(0) {
        if (other.size > 0) {
            data = new T[other.capacity];  // May throw (allocation)
            capacity = other.capacity;

            try {
                for (std::size_t i = 0; i < other.size; ++i) {
                    data[i] = other.data[i];  // May throw (copy)
                }
                size = other.size;
            } catch (...) {
                delete[] data;
                data = nullptr;
                capacity = 0;
                throw;  // Rethrow after cleanup
            }
        }
    }

    // Copy assignment with strong exception guarantee (copy-and-swap)
    Vector<T>& operator=(const Vector<T>& other) {
        if (this != &other) {
            Vector<T> temp(other);  // May throw, but *this unchanged
// ...continues on next slide
```

STL-like Custom Vector Implementation

Code Examples

📌 4. Exception-Safe Vector with Copy-and-Swap (Advanced) (3/4)

```
// ...continued from previous slide
            swap(temp);             // noexcept (just pointer swaps)
        }
        return *this;
    }

    // Move constructor (noexcept)
    Vector(Vector<T>&& other) noexcept : data(nullptr), size(0), capacity(0) {
        swap(other);
    }

    // Move assignment (noexcept)
    Vector<T>& operator=(Vector<T>&& other) noexcept {
        if (this != &other) {
            swap(other);
        }
        return *this;
    }

    void push_back(const T& value) {
        if (size == capacity) {
            std::size_t new_cap = capacity == 0 ? 1 : capacity * 2;
            T* new_data = new T[new_cap];

            try {
                for (std::size_t i = 0; i < size; ++i) {
                    new_data[i] = std::move(data[i]);
                }
            } catch (...) {
                delete[] new_data;
                throw;
            }

            delete[] data;
            data = new_data;
            capacity = new_cap;
        }
        data[size++] = value;
    }

    std::size_t get_size() const { return size; }
// ...continues on next slide
```

STL-like Custom Vector Implementation

Code Examples

📌 4. Exception-Safe Vector with Copy-and-Swap (Advanced) (4/4)

[Code continues across slides]

```
// ...continued from previous slide
};

int main() {
    Vector<ThrowOnCopy> v1;
    v1.push_back(ThrowOnCopy{1});
    v1.push_back(ThrowOnCopy{2});

    Vector<ThrowOnCopy> v2;
    v2.push_back(ThrowOnCopy{10});

    ThrowOnCopy::copy_count = 0;

    try {
        v2 = v1;  // Will throw on 3rd copy
    } catch (const std::exception& e) {
        std::cout << "Exception caught: " << e.what() << "\n";
        std::cout << "v2 size (should be 1, unchanged): " << v2.get_size() << "\n";
        // Output: v2 size (should be 1, unchanged): 1
        // ✅ Strong exception guarantee: v2 remains in original state
    }

    return 0;
}
```

STL-like Custom Vector Implementation

Code Examples

📌 5. Real-World LiDAR Point Cloud Processing (1/4)

This real-world example demonstrates:
- **Move-only semantics**: Disabled copy for large sensor buffers (millions of points)
- **reserve() optimization**: Preallocate capacity to avoid reallocations during 10Hz processing
- **Cache-friendly data**: Contiguous storage with 16-byte aligned points for SIMD
- **Two-stage processing**: Raw → Filtered pipeline typical in autonomous systems

Performance characteristics:
- 100K points × 16 bytes = 1.6 MB per frame
- At 10 Hz LiDAR rate: 16 MB/s data throughput
- Zero reallocations due to `reserve()` → deterministic latency
- Sequential memory access → excellent CPU cache utilization

```cpp
#include <iostream>
#include <cmath>

// High-performance Vector for real-time sensor processing
template <typename T>
class Vector {
private:
    T* data;
    std::size_t size;
    std::size_t capacity;

public:
    Vector() : data(nullptr), size(0), capacity(0) {}
    ~Vector() { delete[] data; }

    Vector(const Vector&) = delete;  // No copy for large sensor data
    Vector& operator=(const Vector&) = delete;

    Vector(Vector&& other) noexcept
        : data(other.data), size(other.size), capacity(other.capacity)
    {
        other.data = nullptr;
        other.size = 0;
        other.capacity = 0;
    }
// ...continues on next slide
```

STL-like Custom Vector Implementation

Code Examples

📌 5. Real-World LiDAR Point Cloud Processing (2/4)

```
// ...continued from previous slide

    void reserve(std::size_t new_cap) {
        if (new_cap <= capacity) return;
        T* new_data = new T[new_cap];
        for (std::size_t i = 0; i < size; ++i) {
            new_data[i] = std::move(data[i]);
        }
        delete[] data;
        data = new_data;
        capacity = new_cap;
    }

    void push_back(T&& value) {
        if (size == capacity) reserve(capacity == 0 ? 1 : capacity * 2);
        data[size++] = std::move(value);
    }

    void clear() { size = 0; }
    std::size_t get_size() const { return size; }
    T& operator[](std::size_t idx) { return data[idx]; }
    const T& operator[](std::size_t idx) const { return data[idx]; }
};

// LiDAR point structure (16 bytes, cache-friendly)
struct alignas(16) Point3D {
    float x, y, z;      // 3D coordinates
    float intensity;    // Reflectivity (0-1)
};

// Autonomous driving: LiDAR processing pipeline
class LidarProcessor {
private:
    Vector<Point3D> raw_cloud;
    Vector<Point3D> filtered_cloud;

public:
    void processFrame(const Point3D* sensor_data, size_t point_count) {
        // Step 1: Load raw data
        raw_cloud.clear();
        raw_cloud.reserve(point_count);  // Avoid reallocations
// ...continues on next slide
```

STL-like Custom Vector Implementation

Code Examples

📌 5. Real-World LiDAR Point Cloud Processing (3/4)

```
// ...continued from previous slide

        for (size_t i = 0; i < point_count; ++i) {
            raw_cloud.push_back(std::move(sensor_data[i]));
        }

        std::cout << "Raw points: " << raw_cloud.get_size() << "\n";

        // Step 2: Filter by distance (keep points within 50m)
        filtered_cloud.clear();
        filtered_cloud.reserve(point_count / 2);  // Estimate half pass filter

        for (size_t i = 0; i < raw_cloud.get_size(); ++i) {
            const Point3D& pt = raw_cloud[i];
            float dist = std::sqrt(pt.x*pt.x + pt.y*pt.y + pt.z*pt.z);

            if (dist < 50.0f && pt.intensity > 0.3f) {
                filtered_cloud.push_back(std::move(raw_cloud[i]));
            }
        }

        std::cout << "Filtered points: " << filtered_cloud.get_size() << "\n";
    }

    const Vector<Point3D>& getFilteredCloud() const {
        return filtered_cloud;
    }
};

int main() {
    // Simulate LiDAR sensor data (100K points per frame)
    constexpr size_t POINTS_PER_FRAME = 100'000;
    Point3D* sensor_data = new Point3D[POINTS_PER_FRAME];

    // Generate random points
    for (size_t i = 0; i < POINTS_PER_FRAME; ++i) {
        sensor_data[i] = {
            static_cast<float>(rand() % 100 - 50),  // x: -50 to 50
            static_cast<float>(rand() % 100 - 50),  // y: -50 to 50
            static_cast<float>(rand() % 10),         // z: 0 to 10
            static_cast<float>(rand() % 100) / 100.0f // intensity: 0-1
// ...continues on next slide
```

STL-like Custom Vector Implementation

Code Examples

📌 5. Real-World LiDAR Point Cloud Processing (4/4)

[Code continues across slides]

```
// ...continued from previous slide
        };
    }

    LidarProcessor processor;
    processor.processFrame(sensor_data, POINTS_PER_FRAME);

    // Output:
    // Raw points: 100000
    // Filtered points: ~33000 (approx 1/3 pass distance+intensity filter)

    delete[] sensor_data;
    return 0;
}
```

What You Learned

Key Takeaways

- ✓ What is a Custom Vector?
- ✓ Why Custom Vector Matters in Autonomous Driving
- ✓ Vector vs Array vs List
- ✓ Iterator Invalidation on Reallocation

→ *Swipe to learn* →